OUTPUT ANALYSIS

Water Depot Network System (Modified) • Graph (Adjacency List), Priority Queue and Dijkstra's Algorithm with Change of Source, Complete Path and Road Update in C++

1. Time Complexity Analysis

Let **V** be the number of locations (water depot, road junctions and repair locations) and **E** be the number of roads currently stored in the graph.

Operation	Time Complexity	Reason
Add Location	**O(log V)**	Duplicate check and insertion in the name map take O(log V); pushing into the names, type and adjacency arrays is O(1) amortised.
Add Road	**O(V)**	Both locations are found by number, name or partial name (worst case a scan of all V locations); the duplicate-road check scans one adjacency list in O(V); the two-way edge is then added in O(1).
Display Locations	**O(V)**	Each location is printed once with its type.
Display Network	**O(V + E)**	Visits every location and every road in the adjacency list once.
Dijkstra (Shortest Route)	**O((V + E) log V)**	Each location is finalised once; each road may push one entry into the min-priority queue, and every push or pop costs O(log V).
Complete Path to One Location (Mod. B)	**O(V)**	Follows parent[] from the location back to the source and reverses it; a path has at most V locations.
Paths to All Locations (Mod. B)	**$O(V^2)$**	pathTo() is called once for every location, and each call costs O(V) in the worst case. For shallow routes it is much smaller.
Find Shortest Route (Option 5)	**$O((V + E) \log V + V^2)$**	One Dijkstra run followed by printing the complete path to every location.
Change Source (Mod. A)	**$O((V + E) \log V + V^2)$**	Dijkstra is run again from the new source and all distances and paths are displayed; the remembered repair location is reused.
Update Road Distance (Mod. C)	**$O((V + E) \log V + V^2)$**	Finding both locations and updating both directions of the road take O(V); the shortest routes are then recalculated from the remembered source.

2. Space Complexity Analysis

Location data (name, type) is stored in vectors and a map, and roads are stored as (neighbour, distance) pairs in an adjacency list. Each road is saved twice because roads are two-way. The results of the last Dijkstra run are kept in dist[] and parent[] so the path can be rebuilt.

Component	Space Complexity	Reason
Total Space Complexity	**O(V + E)**	The graph stores V names and types plus 2E adjacency entries (a road is saved in both directions).
Adjacency List	**O(V + E)**	One list per location; each two-way road adds one entry to each end, so 2E entries in total.
dist[] and parent[]	**O(V)**	Stored inside the network, one value per location, and reused when the source or a road distance changes.
Priority Queue	**O(E)**	Lazy insertion can leave up to one entry per road in the queue at the same time.
Auxiliary Space per Operation	**O(V)**	Rebuilding a path needs an array and a string of at most V locations (one path at a time); add, search and display operations use only O(1) extra variables.

3. Output Observations

The sample session was run on the preloaded network (source: Sangamner Water Depot, repair location: Samnapur).

Test	Input	Result	Observation
Shortest route (Option 5)	Source 1, repair location samnapur	13 km via Sangamner Water Depot -> Bus Stand Chowk -> Pune Naka -> Samnapur	The complete route is displayed (Mod. B) along with the path to every other location.
Update road (Option 7)	Bus Stand Chowk - Pune Naka changed from 6 km to 1 km	Samnapur 13 -> 8 km; Pune Naka 10 -> 5 km; Ghulewadi Junction 11 -> 9 km; MIDC Area 17 -> 13 km; Amrutvahini College 19 -> 17 km	Shortening one road changes the distance to every location that uses it, and the program recalculates automatically (Mod. C).
Change source (Option 6)	New source pune naka	Samnapur 3 km; MIDC Area 8 km; Amrutvahini College 12 km	The remembered repair location is reused and all distances are recalculated from the new source (Mod. A).
Invalid repair location	Source 1, repair location hospital	Error: Invalid source or repair location	Unknown locations are rejected before Dijkstra runs.
Destination not a repair location	Source 1, repair location nashik	Error: Nashik Naka is not a repair location	Only locations of type repair are accepted as destinations.

Interpretation: Adding and displaying operations only touch the data structure once or scan it linearly, so they are cheap. The main cost is Dijkstra's algorithm at O((V + E) log V), because the min-priority queue picks the nearest unvisited location in O(log V); this is faster than scanning an array for the minimum, which costs $O(V^2)$. The three modifications reuse the same algorithm: changing the source (A) and updating a road distance (C) simply run Dijkstra again, so each costs the same as a fresh search, and displaying complete paths (B) adds at most $O(V^2)$ work in the worst case because every path is rebuilt from parent[]. Because all road distances are non-negative, including after an update, the result is always the true least-cost route, and the adjacency list keeps memory linear at O(V + E).